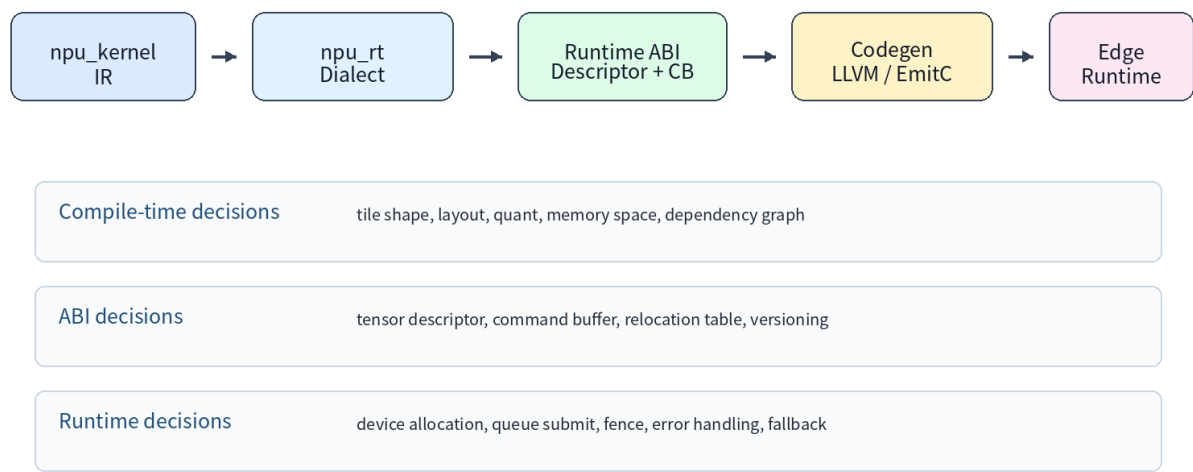


MLIR NPU Compiler - Lecture 19

Runtime / ABI / Codegen

Command buffer 와 tensor descriptor ABI 를 중심으로 compiler backend 와 edge runtime 의 계약을 설계합니다.

Lecture 19 Runtime / ABI / Codegen Pipeline



Lecture Overview

Item	Content	Output
Topic	Runtime / ABI / Codegen	NPU runtime ABI draft
Core IR	npu_rt pseudo dialect, tensor descriptor, command buffer	ABI boundary checklist
Codegen	LLVM dialect, EmitC, binary artifact, hybrid flow	Lowering path decision table
Runtime	C ABI, queue submit, fence, capability, fallback	Runtime API header sketch
Test	FileCheck, manifest validation, simulator, negative diagnostics	Regression matrix

Key Terms

Term	Meaning
------	---------

Tensor descriptor	runtime 이 tensor buffer 를 해석하기 위한 shape/stride/dtype/layout/memory/quant metadata.
Command buffer	NPU/firmware 가 실행할 DMA/compute/barrier packet sequence.
Relocation table	runtime allocation 이후 command buffer 또는 descriptor 안의 address field 를 patch 하기 위한 table.
Fence	host-visible completion object. command 내부 barrier 와 구분해 야 함.
npu_rt dialect	runtime-visible artifact 를 테스트 가능하게 표현하기 위한 교육용 pseudo MLIR dialect.

1. 19 강의 목적

18 강에서 custom NPU dialect 로 npu.matmul, npu.dma, npu.barrier 를 설계했다면, 19 강은 이 IR 을 실제 runtime 이 실행할 수 있는 artifact 로 바꾸는 단계입니다. 이 시점부터는 compiler 내부 표현보다 ABI 안정성이 더 중요합니다.

Runtime ABI 는 단순 함수 호출 규약이 아닙니다. Tensor descriptor, command buffer, artifact manifest, descriptor table, relocation table, error/status code, capability query 까지 포함하는 제품 수준의 계약입니다.

Edge/Embedded NPU 에서는 JIT 보다 AOT artifact, deterministic latency, 제한된 dynamic allocation, firmware/runtime version compatibility 가 중요합니다. 따라서 ABI schema 가 모호하면 성능 문제보다 더 심각한 제품 호환성 문제가 발생합니다.

2. Compiler 와 Runtime 책임 분리

Compiler 는 모델 그래프를 분석해 fusion, layout, tile size, quantization, memory space, command dependency 를 결정합니다. Runtime 은 이미 결정된 artifact 를 device allocation, queue submit, fence/wait, profiling, error handling 의 관점에서 실행합니다.

Runtime 이 compiler decision 을 다시 추론하게 만들면 안 됩니다. 예를 들어 descriptor 에 layout label 만 넣고 stride 를 생략하면 runtime 이 address generation 을 재해석해야 합니다. 이는 target 별 divergence 와 firmware bug 의 원인이 됩니다.

반대로 compiler 가 runtime-only state 를 IR 에 hard-code 해도 안 됩니다. 예를 들어 실제 device address 는 runtime allocation 이후 결정되므로 command buffer 는 relocation table 이나 descriptor index 를 통해 patch 되어야 합니다.

3. Tensor Descriptor ABI

Tensor descriptor 는 runtime 이 tensor buffer 를 해석하기 위한 최소 metadata 입니다. 일반적으로 magic/version, rank, shape, stride, dtype, layout, memory space, base address, byte offset, quantization metadata, flags 를 포함합니다.

Shape 는 logical shape 이고 stride 는 physical memory traversal 을 정의합니다. NPU backend 에서는 vectorization 이나 packing 때문에 logical layout 과 physical layout 이 달라질 수 있습니다. 따라서 stride 와 packing rule 을 명시해야 합니다.

INT8/INT4 quantization 에서는 scale, zero point, quantization axis, storage type, expressed type 이 필요합니다. Per-channel quantization 은 scale array 의 descriptor index 또는 constant table offset 을 추가로 요구할 수 있습니다.

Tensor Descriptor ABI Layout

magic/version	0x4E505544 / ABI v1
rank/dtype/layout	rank=4, dtype=i8, layout=NHWC
shape[0..7]	N,H,W,C plus padding
stride[0..7]	physical addressing stride
base/offset	device address + byte offset
memory_space	DRAM/SRAM/ACCUM/CONST
quant	scale, zero point, axis
flags	readonly, packed, host visible

NPU rule: descriptor is runtime-visible ABI; compiler must not leak unstable IR-only assumptions into this format.

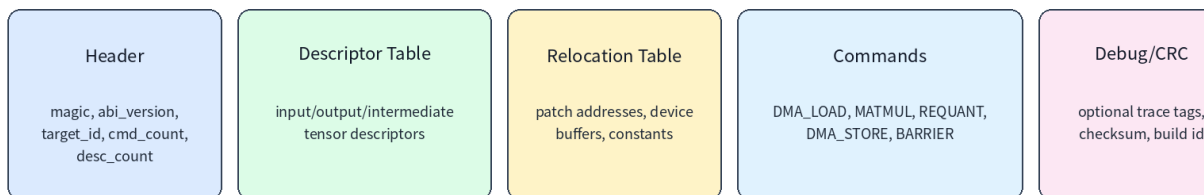
4. Command Buffer ABI

Command buffer 는 NPU 가 실행할 packet sequence 입니다. 기본 구조는 header, descriptor table, relocation table, command packet array, optional debug/profiling block 입니다.

Command header 에는 magic, ABI version, target ID, command count, descriptor count, byte size, alignment, checksum, feature flags 가 들어갑니다. Header 가 없는 raw packet stream 은 디버깅과 versioning 이 어렵습니다.

Command packet 은 DMA_LOAD, DMA_STORE, MATMUL, CONV, ELEMENTWISE, REQUANT, BARRIER, WAIT, SIGNAL, FENCE 등으로 나눌 수 있습니다. 각 packet 은 descriptor index 와 dependency token 을 통해 data movement 와 compute order 를 표현합니다.

Command Buffer Format



Execution order and dependencies



5. Dependency Token 과 Synchronization

고성능 NPU 에서는 DMA engine 과 compute array 가 병렬로 동작합니다. 단순 linear command stream 은 overlap 가능성을 표현하기 어렵기 때문에 token/fence 기반 dependency model 이 필요합니다.

DMA_LOAD 가 token t0 를 produce 하고 MATMUL 이 t0 를 wait 하면, runtime scheduler 는 그 외 독립 DMA 를 overlap 할 수 있습니다. BARRIER 는 command graph 내부 synchronization 이고 FENCE 는 host-visible completion boundary 로 구분해야 합니다.

Verifier 는 unsatisfied token, duplicate token, cyclic dependency, missing fence, illegal memory space transfer 를 잡아야 합니다.

6. npu_rt Dialect 설계

교육용 pseudo dialect 인 npu_rt 는 runtime-visible artifact 를 표현하기 위한 마지막 MLIR layer 입니다. npu_rt.tensor_desc 는 descriptor table entry 를 만들고, npu_rt.command_buffer 는 command sequence region 을 표현합니다.

npu_rt.submit, npu_rt.wait, npu_rt.call_runtime 같은 op 는 host runtime API call 로 lowering 될 수 있습니다. 또는 npu_rt.emit_artifact 가 binary command buffer 와 manifest 를 직접 생성하도록 설계할 수도 있습니다.

npu_rt dialect 는 upstream MLIR dialect 가 아니라 custom backend 설계 예제입니다. 핵심은 MLIR 의 progressive lowering 관점에서 runtime ABI boundary 를 IR 로 명시해 testable 하게 만드는 것입니다.

7. Codegen 선택지: LLVM IR, EmitC, Binary Artifact

LLVM dialect 경로는 host stub, prototype, JIT-like flow, C-compatible wrapper 생성에 유리합니다. MLIR 의 LLVM IR target 문서는 MLIR 을 LLVM dialect 로 변환한 후 LLVM IR 로 translate 하는 two-stage flow 를 설명합니다.

EmitC 경로는 embedded static build 에 유리합니다. npu_rt.submit 을 emitc.call_opaque 로 낮추고, descriptor 와 command buffer 를 C array initializer 로 만들어 cross-compile 할 수 있습니다.

Binary artifact 경로는 runtime submit path 가 가장 단순합니다. compiler 가 command_buffer.bin 과 manifest.yaml 을 생성하고, runtime 은 이를 load/relocate/submit 합니다. 실제 제품에서는 C stub + binary command buffer + manifest 의 hybrid 구성이 실용적입니다.

Codegen Choices for Edge NPU Runtime

LLVM Dialect -> LLVM IR	native host stubs, JIT/prototype, C-compatible wrappers, memref descriptor lowering
EmitC -> C/C++	embedded-friendly generated C, static build, easy cross compile
Binary Command Buffer	no host codegen needed, compile-time artifact submitted to runtime
Hybrid	C runtime stub + binary command buffer + metadata manifest

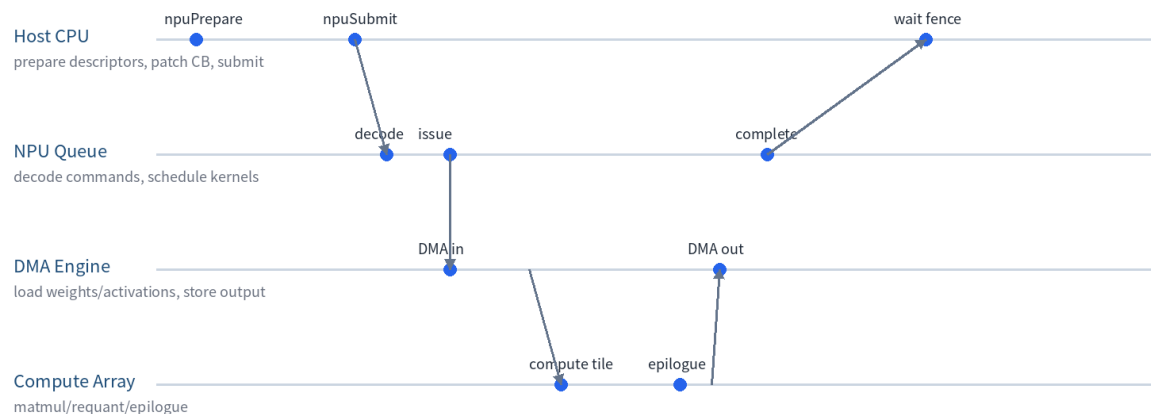
8. Runtime C ABI 설계

C ABI 는 C++ name mangling 과 ABI compatibility 문제를 줄이는 데 유리합니다. NpuContext, NpuModule, NpuQueue, NpuCommandBuffer, NpuFence 같은 opaque handle 을 사용하고, API 는 fixed-width type 으로 정의합니다.

API 예시는 npuInit, npuCreateContext, npuLoadModule, npuBindTensor, npuSubmit, npuWait, npuGetStatus, npuQueryCapability 입니다. Ownership 과 lifetime, thread-safety, error handling, timeout semantics 를 문서화해야 합니다.

Runtime API 는 compiler 가 만든 artifact 의 version 을 검사하고, target feature 가 맞지 않으면 명확한 status code 를 반환해야 합니다.

Runtime Execution Timeline



9. Testing 과 Debugging

Runtime ABI 는 text IR 만으로 충분히 검증하기 어렵습니다. Descriptor manifest, command buffer schema, FileCheck test, golden simulator, binary dump tool 을 함께 사용해야 합니다.

FileCheck 는 `npu_rt` lowering 이 descriptor count, command kind, dependency token, runtime call signature 를 올바르게 생성 하는지 검사합니다. Negative test 는 unsupported dtype, incompatible layout, unaligned buffer, missing fence 를 포함해야 합니다.

Debugging 을 위해 descriptor dump, command trace, firmware log, compiler debug symbol map 을 연결해야 합니다. 가장 좋은 버그 리포트는 최소 command buffer 와 manifest 를 포함합니다.

10. 20 강 Capstone 으로 연결

19 강 산출물은 20 강 capstone 의 runtime/codegen 장이 됩니다. 20 강에서는 frontend 부터 runtime artifact 까지 end-to-end mini NPU compiler design spec 을 작성합니다.

Capstone 에서는 tensor descriptor ABI, command buffer schema, runtime API, fallback policy, codegen path, regression test matrix 를 하나의 문서로 통합해야 합니다.

Recommended npu_rt Lowering Pipeline

```
// pseudo pass pipeline for Lesson 19
builtin.module(
  npu-kernel-to-npu-rt,
  npu-rt-verify-abi,
  npu-rt-pack-command-buffer,
  npu-rt-materialize-descriptor-table,
  npu-rt-emit-artifact-manifest,
  canonicalize,
  cse
```

```

)

// host stub option
builtin.module(
  convert-npu-rt-to-emitc,
  canonicalize
)

// LLVM host wrapper option
builtin.module(
  convert-npu-rt-to-llvm,
  finalize-memref-to-llvm,
  convert-func-to-llvm,
  reconcile-unrealized-casts
)

```

Runtime ABI Checklist

- ABI major/minor version and target ID are present.
- All structs use fixed-width integer types and explicit alignment.
- Tensor descriptor has shape, stride, dtype, layout, memory space, quant metadata.
- Command buffer has header, descriptor table, relocation table, command packet array.
- Every command has dependency token semantics or is explicitly sequential.
- Host-visible fence and command-internal barrier are separated.
- Capability query and fallback path are documented.
- Positive and negative tests cover ABI legality.

References

MLIR LLVM IR Target: <https://mlir.llvm.org/docs/TargetLLVMIR/>

MLIR LLVM Dialect: <https://mlir.llvm.org/docs/Dialects/LLVM/>

MLIR EmitC Dialect: <https://mlir.llvm.org/docs/Dialects/EmitC/>

MLIR C API: https://mlir.llvm.org/docs/C_API/

MLIR Pass Infrastructure: <https://mlir.llvm.org/docs/PassManagement/>